

NLP 序列标注任务详解

本文档面向 NLP 算法工程师新手，系统梳理序列标注任务的核心知识点。文中对专业术语和晦涩概念均做了inline注解或统一收录在附录术语表中。

第一章 序列标注基础

1.1 核心定义

什么是序列标注？

序列标注 (Sequence Labeling) 是自然语言处理中的一种核心任务范式。其本质是：

给定一个输入序列，为序列中的每一个元素分配一个标签，输出序列与输入序列等长。

用数学形式表达：

$$X = (x_1, x_2, \dots, x_n) \rightarrow Y = (y_1, y_2, \dots, y_n)$$

其中：

- X 是输入序列 (如一个句子中的 **token** 序列。*token* 是文本的最小处理单元，可以是字、词或子词，取决于分词方式)
- Y 是输出标签序列
- 输入与输出严格等长 (n to n 的映射关系)

🔗 **新手直觉类比**：把序列标注想象成给一篇文章的每个字标上拼音声调。输入是"我爱北京天安门"这7个字，输出也是7个字对应的声调 (如"我=3声, 爱=4声, 北=3声, ...))，输入和输出一一对应，不增不减。NER、分词本质上就是在做类似的事——只是把"声调"换成了"实体类型"或"词边界标记"。

【深入理解】Token — 文本的最小处理单元

Token 是 NLP 中最基础的概念，但不同场景下含义不同：

Token 类型	示例 (文本: "我爱北京天安门")	说明
字级 Token	我 / 爱 / 北 / 京 / 天 / 安 / 门	每个汉字一个 token，最细粒度，适合中文 NER、分词
词级 Token	我 / 爱 / 北京 / 天安门	先分词再建模，token 就是词语
子词 Token (WordPiece/BPE)	我 / 爱 / 北 / ##京 / 天 / ##安 / ##门	BERT 默认方式，罕见词拆成子词，避免 OOV

新手要点：本文讨论的"序列标注"默认基于**字级 Token**——每个汉字是一个 token，模型为每个汉字预测一个标签。这样做的好处是模型可以从零开始学习切词和实体边界，不依赖预分词结果。

直观示例

示例 1：命名实体识别（NER）

输入	马云	创立	了	阿里巴巴
标签	B-PER	O	O	B-ORG / I-ORG

- "马云"被识别为人名（PER），"马"是实体起始（B-PER），"云"是实体内部（I-PER）
- "阿里巴巴"被识别为机构名（ORG），"阿"为起始（B-ORG），"里巴巴"为内部（I-ORG）
- "创立"和"了"为非实体（O）

示例 2：中文分词

输入	我	爱	北	京	天	安	门
标签	S	S	B	E	B	I	E

- "我"和"爱"单字成词（S）
- "北京"二字词：北为词首（B），京为词末（E）
- "天安门"三字词：天为词首（B），安为词中（I），门为词末（E）

1.2 与分类任务的区别

序列标注与传统的文本分类虽然都属于**监督学习**（即从标注好的数据中学习输入到输出的映射关系）任务，但在输出粒度、输出长度和任务性质上有根本差异：

维度	文本分类	序列标注
输出粒度	一个序列一个标签	每个 token 一个标签
输出长度	固定为 1	与输入等长（n 个）
典型任务	情感分析、主题分类	NER、分词、词性标注
标签间关系	无（独立标签）	有依赖关系（BIO 约束）
上下文依赖	整体语义即可	局部上下文至关重要

详细对比说明

1. 输出粒度不同：

- 文本分类对整段文本输出一个全局标签，例如"这段评论是正面的"
- 序列标注对序列中每个位置都输出一个标签，例如"第3个字是地名实体的开头"

2. 输出长度不同：

- 文本分类无论输入多长，输出始终是单个标签
- 序列标注的输出长度严格等于输入长度，输入 n 个 token 就输出 n 个标签

3. 标签间依赖不同：

- 文本分类的标签是独立的，不存在标签之间的约束
- 序列标注的标签之间存在强依赖关系，例如 **I-PER** 不能出现在 **B-LOC** 之后，**I** 标签不能出现在序列开头

4. 上下文利用方式不同：

- 文本分类主要关注全局语义
- 序列标注中每个位置的预测都高度依赖局部上下文（前后若干 token 的信息），因为同一个词在不同上下文中可能有不同的标签

1.3 序列标注的三个关键特征

特征一：输入与输出严格等长（n to n）

序列标注的核心约束是输入序列和输出序列的长度完全一致。每一个输入元素（token）都恰好对应一个输出标签，不存在多对一或一对多的映射。

这意味着：

- 模型需要对每个位置做独立但有关联的决策
- 不像 Seq2Seq（Sequence-to-Sequence，序列到序列任务，如机器翻译，输入输出长度可以不同）任务那样输入输出长度可以不同
- 这种等长特性使得序列标注可以建模为逐位置的分类问题，但需要考虑标签间的依赖

特征二：标签之间存在依赖关系（BIO 约束）

序列标注的标签不是**独立同分布**（即每个标签的出现概率互不影响）的，相邻标签之间有严格的约束关系。

🧠 **新手类比**：把 BIO 标签想象成**交通规则**——你开车时不能随意切换路线，有些移动是“违章”的。比如你不能从“直行车道(O)”突然变成“左转车道(B-PER)”而不经“转向灯(I-PER)”。CRF 的作用就是一个**学会了所有交通规则的司机**，它知道哪些路径是合法的，会主动避开违章路线。

BIO 标签的合法转移“路线图”

把每个位置的标签决策看成一幅**状态转移图**，下图中 ✓ 表示合法转移，✗ 表示非法转移：

上一个位置的标签 → 当前可选标签

B-PER	——✓——→	I-PER	（继续同一个人名实体，合法）
	——✓——→	O	（人名结束了，合法）
	——✗——→	I-ORG	（人名后面不能紧跟机构名内部！类型不一致）
	——✗——→	B-ORG	（人名后可以接机构名起始，这是合法的✓）
I-PER	——✓——→	I-PER	（人名内部继续，合法）
	——✓——→	O	（人名结束，合法）
	——✗——→	I-ORG	（类型不一致，非法）
	——✓——→	任何 B-* 或 O	（合法）
O	——✓——→	B-PER	（开始一个新的人名，合法）
	——✓——→	B-ORG	（开始一个新的机构名，合法）
	——✓——→	O	（继续非实体，合法）
	——✗——→	I-PER	（不能凭空出现“实体内部”！必须先有B！）
	——✗——→	I-ORG	（同上）

BIO 约束的三大规则清单

规则	含义	错误示例	为什么错
起始规则	I-X 标签不能出现在序列第一个位置	句子开头就是 I-PER	I 表示"实体内部", 开头必须是 B 或 O
类型一致规则	I-X 前一个标签必须是 B-X 或 I-X (同类)	B-PER → I-ORG	人名实体不能突然变成机构名实体
O 跳转规则	I-X 前一个标签不能是 O	O → I-PER	非实体后面不能突然出现"实体内部"

这些约束源于标注体系的语义定义，也是 CRF（条件随机场）等模型发挥作用的根本原因——CRF 可以学习并强制执行这些转移约束。

特征三：上下文信息对预测至关重要

同一个词在不同上下文中可能具有不同的标签：

- "苹果"在"我吃了一个**苹果**"中是食物（或普通名词）
- "苹果"在"**苹果**发布了新款 iPhone"中是公司名（ORG）

因此，序列标注模型必须充分利用上下文信息：

- BERT** (*Bidirectional Encoder Representations from Transformers*，基于双向 *Transformer* 的预训练语言模型，能同时获取左侧和右侧上下文) 通过自注意力机制捕捉全局上下文
- LSTM** (*Long Short-Term Memory*，长短期记忆网络，一种能记忆长距离依赖的循环神经网络) 的隐藏状态能编码历史信息
- 上下文信息的有效利用是序列标注模型性能的关键因素

1.4 序列标注在 NLP 中的地位

序列标注是 NLP 中最基础、最核心的任务范式之一。许多重要的 NLP 任务都可以转化为序列标注问题：

序列标注任务家族：

- ├─ 中文分词 (Tokenization)
- ├─ 词性标注 (POS Tagging)
- ├─ 命名实体识别 (NER)
- ├─ 语义角色标注 (SRL)
- ├─ 关键词/短语抽取
- └─ 事件抽取 (Event Extraction)

这些任务的共同本质是：定义标注体系 → 标注训练数据 → 训练序列标注模型 → 解码输出。因此，掌握序列标注的基本原理，就掌握了解决这一大类 NLP 任务的通用方法论。

第二章 NLP 中的典型应用

序列标注在 NLP 领域有广泛的应用场景，本章介绍四个典型的序列标注任务：中文分词、词性标注、命名实体识别以及其他序列标注应用。

2.1 中文分词 (Tokenization)

任务定义

中文分词是将连续汉字序列切分为有意义的词语单元的过程，是中文 NLP 的基础预处理步骤。

与英文不同，中文文本没有天然的空格分隔符，因此分词是中文 NLP 的第一步，也是后续所有高级任务（词性标注、命名实体识别、句法分析等）的基础。

标注示例

以"我爱北京天安门"为例：

字	我	爱	北	京	天	安	门
BIES 标签	S	S	B	E	B	I	E
分词结果	我	爱	北京	天安门			

BIES 标注体系

BIES 是中文分词中最常用的标注体系，包含四个标签：

标签	含义	说明
B	Begin	词语的起始字
I	Inside	词语的中间字
E	End	词语的结束字
S	Single	单字成词

标注逻辑：

- 单字词（如"我"、"爱"）标注为 S
- 多字词的第一个字标注为 B，最后一个字标注为 E，中间字标注为 I
- 通过 BIES 序列可以无歧义地还原分词结果

BI 简化体系

BI 体系是 BIES 的简化版本：

- B = 词首
- I = 词内其余字（无 E/S 区分）

BI 体系更简洁，标签数更少，但在边界识别上不如 BIES 精确。

为什么用字符级标注？

1. **中文无天然分隔符**：英文天然以空格分词，中文必须通过算法切分。字符级建模提供了一个统一的框架，每个汉字对应一个标签
2. **新词/未登录词 (OOV) 处理更鲁棒**：OOV (*Out-of-Vocabulary*，未登录词，即词典中未收录的词)。基于词典的分词方法遇到词典中没有的词就会失败，而字符级序列标注模型可以通过上下文推断新词边界
3. **与 NER 等任务共享相同框架**：分词和 NER 都是序列标注任务，可以在同一模型框架下统一处理

Python 示例 (jieba 分词对比)

```
import jieba

words = jieba.lcut('我爱北京天安门')
# ['我', '爱', '北京', '天安门']
```

jieba 是基于词典的分词工具，速度较快但处理新词能力有限。序列标注模型则能更好地处理 OOV 问题。

2.2 词性标注 (POS Tagging)

任务定义

词性标注 (Part-of-Speech Tagging) 是为句子中每个词 (或字) 分配其在该上下文中的语法类别 (词性)，是句法分析的基础。

词性标注的核心挑战是**词性消歧**：同一个词在不同上下文中可能有不同的词性。例如"打"在"打篮球"中是动词，在"一打啤酒"中是量词。

标注示例

输入	马云	创立	了	阿里巴巴	公司
词性	nr	v	u	nz	n
说明	人名	动词	助词	专名	名词

常见词性标签 (PKU 标注体系)

PKU (北京大学) 标注体系是中文词性标注的标准之一：

标签	词性	示例
n	名词	北京、公司
v	动词	创立、发展
a	形容词	优秀、快速
d	副词	非常、已经
nr	人名	马云、李明
ns	地名	北京、上海
nz	其他专名	阿里巴巴
u	助词	了、的、地

与分词的关系

词性标注与分词密切相关，构成 NLP 流水线的前两个步骤：

分词 → 词性标注 → NER / 句法分析
(典型 NLP 流水线前处理)

- **联合建模**：实际系统中常将分词和词性标注联合建模，同步输出，避免错误传播
- **结构化特征**：词性为下游任务提供结构化特征，例如 NER 系统可以利用词性信息提高识别准确率
- **BERT 时代的变化**：在 BERT 等预训练模型时代，词性标注可以作为序列标注**微调** (*Fine-tuning*，在预训练模型基础上用下游任务数据继续训练，使模型适应特定任务) 任务，也可以作为下游任务的辅助特征

2.3 命名实体识别 (NER)

任务定义

命名实体识别 (Named Entity Recognition, NER) 是从文本中识别出具有特定意义的实体，包括人名、地名、机构名等，是信息抽取的核心任务。

BIO 标注体系

BIO 是 NER 中最基础的标注体系：

标签	含义
B-TYPE	实体的起始 token (B = Begin)
I-TYPE	实体的内部 token (I = Inside)
O	非实体 token (O = Outside)

BIO 标注示例：

字	马	云	创	立	阿	里	巴	巴
标签	B-PER	I-PER	O	O	B-ORG	I-ORG	I-ORG	I-ORG

BIOES 标注体系

BIOES 是 BIO 的扩展，增加 E (End) 和 S (Single) 标签：

标签	含义
B-TYPE	实体的起始 token
I-TYPE	实体的内部 token
O	非实体 token
E-TYPE	实体的结束 token
S-TYPE	单 token 实体

BIOES 标注示例：

字	马	云	创	立	阿	里	巴	巴
标签	B-PER	E-PER	O	O	B-ORG	I-ORG	I-ORG	E-ORG

BIOES 的优势：

- **边界更明确**：E 标签明确标记实体的结束位置
- **单字实体区分更清晰**：S 标签专门标记单 token 实体
- **标签更多**：但标签数量翻倍，标注更复杂

常见实体类型（CoNLL 标准）

CoNLL（*Conference on Computational Natural Language Learning*，计算自然语言学习会议，是 NLP 领域重要的学术会议，其共享任务定义的标注标准被广泛使用）定义了四种标准实体类型：

类型	标签	示例
人名	PER	马云、李明
地名	LOC	北京、上海
机构名	ORG	阿里巴巴、清华大学
其他专名	MISC	iPhone 15、NBA

2.4 序列标注的其他 NLP 应用

除了分词、词性标注和 NER 这三大经典任务，序列标注范式还能统一处理许多其他 NLP 任务。本节逐一深入解析语义角色标注、依存句法分析（简化视角）、关键词抽取和事件抽取。

2.4.1 语义角色标注（SRL）

任务定义

语义角色标注（Semantic Role Labeling, SRL）用于标注**谓词-论元结构**，通俗地说就是回答**"谁对谁、做了什么、在哪里、什么时候"**这套问题。它是连接句法分析和深层语义理解的桥梁。

● **新手类比**：SRL 就像给你的句子做"六要素"拆解——一条新闻的"5W1H"（Who、What、Whom、Where、When、How）。SRL 模型就是自动帮你把每个句子的这些要素标注出来。

核心标签体系

SRL 的标注围绕**谓词（Predicate）**展开，谓词通常是句子的核心动词或事件名词。每个谓词支配一组**论元（Argument）**：

标签	全称	含义	提问方式
V	Verb/Predicate	谓词，句子的核心动作或事件	"发生了什么？"
ARG0	Agent	施事者，主动发出动作的一方	"谁做的？"
ARG1	Patient/Theme	受事者，承受动作或受动作影响的一方	"对谁/什么做的？"
ARG2	Benefactive/Instrument	受益者、工具、间接宾语等	"为谁？/用什么？"
ARG3	Start point	起点、来源	"从哪里来？"
ARG4	End point	终点、目标	"到哪里去？"
ARGM-TMP	Temporal	时间修饰	"什么时候？"
ARGM-LOC	Location	地点修饰	"在哪里？"
ARGM-MNR	Manner	方式修饰	"怎么样做？"

ARG 是核心论元（编号越小越核心），ARGM 是附加修饰论元（以 - 后缀标识修饰类型）。

完整标注示例

例句："马云昨天在杭州正式创立了阿里巴巴。"

字	马	云	昨	天	在	杭	州	正	式	创	立	了	阿	里	巴	巴
SRL 标签	B-ARG0	I-ARG0	B-ARGM-TMP	I-ARGM-TMP	B-ARGM-LOC	I-ARGM-LOC	I-ARGM-LOC	B-ARGM-MNR	I-ARGM-MNR	B-V	I-V	O	B-ARG1	I-ARG1	I-ARG1	I-ARG1

从这个标注中，我们可以读出结构化信息：

谓词（V）	：创立	
施事（ARG0）	：马云	← 谁创立的？
受事（ARG1）	：阿里巴巴	← 创立了什么？
时间（ARGM-TMP）	：昨天	← 什么时候？
地点（ARGM-LOC）	：杭州	← 在哪里？
方式（ARGM-MNR）	：正式	← 怎么样创立的？

多谓词场景

一个句子可以包含多个谓词，SRL 会围绕每个谓词**独立标注一遍**：

例句："马云创立阿里巴巴后，投资了很多初创企业。"

以谓词"创立"视角：

字	马	云	创	立	阿	里	...
标签	B-ARG0	I-ARG0	B-V	I-V	B-ARG1	I-ARG1	...

以谓词"投资"视角：

字	...	投	资	了	很	多	初	创	企	业
标 签	...	B- V	I- V	O	B- ARG1	I- ARG1	I- ARG1	I- ARG1	I- ARG1	I- ARG1

注意"马云"在"投资"视角下仍然是施事 ARG0（承前省略），模型需要跨越逗号识别这种长距离依赖。

SRL 的业务价值

应用场景	说明	示例
信息抽取	从新闻中提取"谁收购了谁"	腾讯(V) ARG0→收购→ARG1 搜狗
知识图谱构建	将非结构化文本转为三元组	(马云, 创立, 阿里巴巴)
智能问答	理解用户问题中的人物/事件关系	"谁创立了阿里巴巴？" → 从 SRL 解析中直接检索
舆情分析	识别事件的主体和客体	"消费者 ARG0→投诉→ARG1 产品质量问题"

新手要点：SRL 的标注粒度比 NER 更细——NER 只管"这是什么实体"，SRL 还要搞清楚"这个实体在这句话里扮演什么角色"。同一个实体在不同句子中可能扮演不同角色（如"马云"有时是 ARG0，有时是 ARG1）。

2.4.2 依存句法分析（简化视角）

任务定义

依存句法分析（Dependency Parsing）的目标是找出句子中**词与词之间的语法依存关系**——哪个词依附于哪个词、依附关系是什么类型。标准做法是输出一棵依存树，但也可以简化为序列标注问题。

🔗 **新手类比：**依存句法分析就像给句子里每个词找一个"老大"（Head）。大部分词都听谓语动词的，而谓语动词自己则是句子的"大 Boss"（ROOT）。标注每个词的"老大是谁"就等价于做序列标注。

简化方案：标注 Head 位置偏移量

将依存句法分析转化为序列标注的核心思路是：**为每个词标注它的 Head 词在句子中的位置索引。**

例句："马云创立阿里巴巴"

先看标准依存分析的结果：

马云 ← 创立 → 阿里巴巴
(nsubj) (dobj)

翻译：

- "马云"的 Head 是"创立"（主谓关系 nsubj）
- "阿里巴巴"的 Head 是"创立"（动宾关系 dobj）
- "创立"的 Head 是 ROOT（句子的根）

转化为序列标注（标注 Head 的位置偏移量）：

词	马云	创立	阿里巴巴
Head 词	创立 (位置 2)	ROOT (位置 0)	创立 (位置 2)
Head 偏移	+1 (向后 1 位)	-0 (无 Head)	-1 (向前 1 位)

偏移量 = Head 位置 - 当前词位置。正数表示 Head 在后面，负数表示 Head 在前面。

完整标注表格：

位置	词	Head	Head 偏移	依存关系 (辅助)
1	马云	创立	+1	nsubj (主谓)
2	创立	ROOT	0	root
3	阿里巴巴	创立	-1	dobj (动宾)

更复杂的例子

例句："我非常喜欢吃北京的烤鸭。"

位置	词	Head 词	Head 偏移	依存关系
1	我	喜欢	+2	nsubj
2	非常	喜欢	+1	advmod (状语)
3	吃	喜欢	-1	xcomp (补语)
4	北京	烤鸭	+3	nmod (名词修饰)
5	的	北京	-1	case (格标记)
6	烤鸭	吃	-3	dobj (动宾)

对应的序列标注 (仅看 Head 偏移)：

输入：我	非常	吃	喜欢	北京	的	烤鸭
标签：+2	+1	-1	0	+3	-1	-3

依存关系类型速查

完整标注中除了 Head 偏移还经常标注关系类型，以下是常见的依赖关系：

标签	全称	含义	示例
nsubj	nominal subject	名词主语	马云 → 创立
dobj	direct object	直接宾语	创立 → 阿里巴巴
nmod	noun modifier	名词修饰语	北京 → 烤鸭
advmod	adverbial modifier	状语修饰	非常 → 喜欢
case	case marker	格标记	北京 ← 的
root	root	句子的根	喜欢 ← ROOT

序列标注方案 vs 标准方法的对比

维度	序列标注方案	标准基于图的解析器
精度	中等 (UAS ~85%)	高 (UAS ~95%)
速度	极快 (一次前向传播)	慢 (需搜索最优树)
实现难度	低 (复用序列标注框架)	高 (需专用解码器)
适用场景	实时系统、轻量部署	离线分析、学术研究

UAS (Unlabeled Attachment Score)：只看 Head 找得对不对，不考虑关系类型。是依存分析的基础指标。

序列标注方案的局限性

- 1. **多 Head 问题**：某些语法理论中一个词可以有多个 Head，序列标注只能输出一个
 - 2. **环路问题**：如果标注出 A→B 且 B→A，就形成非法环路
 - 3. **Non-projective 结构**：某些语言（如捷克语、德语）存在交叉依存弧，偏移量方案无法表示
- 新手要点**：你的项目如果是中文/英文的在线实时分析场景（如搜索引擎 query 解析、聊天机器人意图理解），序列标注式依存分析是个高性价比的选择。如果是离线高精度研究分析，还是建议用标准 biaffine parser 或基于图的解析器。

2.4.3 关键词/短语抽取

任务定义

关键词/短语抽取 (Keyword/Keyphrase Extraction) 是从一段文本中自动提取最能代表其核心内容的关键词语，广泛用于文档摘要、搜索索引、内容推荐等场景。

🕒 **新手类比**：关键词抽取就像你读一篇长文章后用荧光笔划重点——荧光笔划的那些词或短语就是"关键词"。序列标注就是训练一个"自动荧光笔"。

序列标注方案

用标准 BIO 标签体系标注关键词 span，然后像 NER 一样训练模型：

标注示例 1 (科技新闻)：

文本：华为今日发布新一代麒麟芯片，性能大幅提升。

字级标注：

	华		为		今		日		发		布		新		一		代		麒		麟		芯		片		,		性		能	
	B-KW		I-KW		O		O		O		O		O		O		B-KW		I-KW		I-KW		I-KW		O		O		O		O	

抽取结果：

- 华为（关键词 span 1）
- 麒麟芯片（关键词 span 2）

标注示例 2（学术论文摘要）：

文本：本文提出了一种基于深度学习的文本分类方法。

字级标注：

	本		文		提		出		了		一		种		基		于		深		度		学		习		的		文		本		分	
	O		O		O		O		O		O		O		O		O		O		B-KW		I-KW		I-KW		I-KW		O		B-KW		I-KW	

抽取结果：

- 深度学习
- 文本分类

与 NER 的微妙区别

维度	NER	关键词抽取
实体来源	预定义的类别（人名、地名...）	上下文决定哪些词重要
实体唯一性	同一个实体出现在任何文本中标签不变	同一个词在不同文章中可能是/不是关键词
标注难度	需要领域知识	标注者主观性更强，一致性更难保证
标签稳定性	稳定（"马云"永远是 PER）	不稳定（"苹果"在某些文档中是关键词，某些不是）

结合置信度的 Top-K 输出

实际应用中，一篇文档可能需要输出 Top-K 个关键词，而不是"所有关键词"。这时可以让模型对每个预测赋予置信度（通常取该 span 各 token 预测概率的均值），然后排序截断：

```
# 伪代码：对每个预测的 keyword span 计算置信度并排序
keywords_with_confidence = []
for span in predicted_spans:
    probs = [model_output[t][label_id] for t in span.tokens] # 各 token 的概率
    confidence = sum(probs) / len(span.tokens) # 平均置信度
    keywords_with_confidence.append((span.text, confidence))

# 取 Top-5
top_k = sorted(keywords_with_confidence, key=lambda x: x[1], reverse=True)[:5]
```

新手要点：关键词抽取的难点在于“重要性”的界定本身就主观，不像 NER 的实体类别那么客观。两个标注者对同一篇文章标注关键词，一致性常低于 70%。因此在项目中务必先和业务方对齐“什么算关键词”的标注指南。

2.4.4 事件抽取 (Event Extraction)

任务定义

事件抽取 (Event Extraction) 是从非结构化文本中识别**发生了什么事、谁参与了、何时何地、结果如何**。它是信息抽取的“终极形态”，比 NER 更进一步——NER 只管“实体是什么”，事件抽取还要回答“实体之间发生了什么”。

🍳 **新手类比：**NER 是认出食材（这是土豆、这是牛肉），事件抽取是看懂一道菜（这是一道红烧牛肉，“炒”是烹饪方式/触发词，牛肉和土豆是食材/论元）。事件抽取需要 NER 作为前提，但比 NER 层次更高。

事件抽取包含两个核心子任务，**本质上都是序列标注**：

子任务 1：触发词识别 (Trigger Detection)

触发词是标志事件发生的核心词语，通常是动词或事件名词。目标：找出文本中所有触发词并分类事件类型。

标注示例：

文本：腾讯于2024年6月以10亿美元收购了搜狗的搜索业务。

触发词识别标注（简化 BIO + 事件类型）：

腾	讯	于	2	0	2	4	年	6	月	以	1	0	亿	美	元	收	购	了	
O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	B-收购	I-收购	O

识别结果：触发词 = "收购"，事件类型 = "收购事件"

常见事件类型（以 ACE 2005 标准为例）：

事件类型	典型触发词	业务含义
Transaction	收购、出售、买入、卖出	资产交易
Movement	前往、抵达、离开	人物移动
Personnel	任命、离职、升迁	人事变动
Business	合并、破产、上市	商业事件
Conflict	袭击、爆炸、逮捕	冲突事件
Justice	起诉、判决、逮捕	司法事件

子任务 2：论元抽取（Argument Extraction）

识别事件触发词之后，还需要找出参与该事件的各个**论元角色**（事件参与者、时间、地点、金额等）。

完整事件抽取标注示例：

沿用上句："腾讯于2024年6月以10亿美元收购了搜狗的搜索业务。"

论元角色	论元内容	含义
收购方（Buyer）	腾讯	谁发起的收购
被收购方（Seller）	搜狗	被收购的对象
收购标的（Target）	搜索业务	收购了什么
收购金额（Price）	10亿美元	花了多少钱
时间（Time）	2024年6月	什么时候

用 BIO 序列标注论元：

触发词"收购"已知，对其标注论元角色：

	腾		讯		于		2		0		2		4		年		6		月		以		1		0		亿		美		元		收		购		了	
	B-Buy				I-Buy				B-Time								B-Price								TRG			TRG			O							

TRG = 触发词标记（特殊标签）

注意：论元标注中"时间""金额"这些也是 span（连续片段），只是标签语义从"这是什么实体"变成了"这在事件中扮演什么角色"。

多事件场景

一个句子可能包含多个事件：

例句："张三因涉嫌诈骗已被警方逮捕，案件正在进一步审理中。"

事件 1 (逮捕)	事件 2 (审理)
触发词: "逮捕"	触发词: "审理"
人物: "张三"	施事: "警方" (隐含)
罪名: "诈骗"	受事: "案件"
施事: "警方"	状态: "进一步"

模型需要同时标出两个触发词，并为每个触发词分别标注论元。

事件抽取的难度层级

难度	子任务	类比	当前水平
★★	触发词识别	"认出句子里的动作词"	较成熟
★★★	单事件论元抽取	"识别一个事件的参与者"	较成熟
★★★★	多事件分离	"一句话中有两个事件，分清各自的论元"	有挑战
★★★★★	跨句事件	"上句说腾讯收购，下句说搜狗回应"	前沿难题

新手要点：事件抽取是目前 NLP 的"高端任务"，不要一上来就全做。建议按照"触发词识别 → 单事件论元抽取 → 多事件"的顺序渐进推进。如果业务只需要"从新闻里提取收购信息"，那就没必要做成通用事件抽取系统，做成针对性的 NER（实体 + 关系）就够了。

2.5 共同本质

所有上述任务都可以归结为同一套方法论：

定义标注体系 → 标注训练数据 → 训练序列标注模型 → 解码输出

不同任务的区别仅在于标注体系的定义（标签的含义和约束不同），而模型架构和训练方法是通用的。这就是序列标注范式的强大之处——掌握一个任务的方法，即可迁移到其他任务。

第三章 NER：泛信息抽取能力

命名实体识别（NER）不仅仅局限于人名、地名、机构名的识别，其本质是一个**通用的 span 分类器**，可以识别任意业务关心的文本片段。

3.1 传统认知的局限

常见误解 vs. 正确理解

常见误解	正确理解
X NER 只能识别人名、地名、机构名	✓ 实体 = 任意业务关心的文本 span
X 实体类型是固定的	✓ 类型由业务需求自由定义
X 只有专有名词才算实体	✓ 可以是术语、属性、数值、事件...
X 不同业务需要完全不同的方法	✓ 处理思路：定义 → 标注 → 训练

核心洞察

NER 的本质是通用 span 分类器，与具体实体类型无关。

只要定义了标签体系并提供足够的标注样本，NER 模型就能识别任意类型的文本片段。模型学习的是"给定上下文，判断一段连续 token 是否属于目标类别"这一通用能力，与类别叫什么名字无关。

3.2 NER 是泛信息抽取框架

框架本质

NER 模型学习的能力是：

给定上下文，判断一段连续 token 是否属于目标类别。

这个能力与类别的名称无关。只要提供足够的标注样本，模型就能识别任意类型的 span。

实体可以是任意类型

不同行业和场景下，实体类型的定义完全不同：

领域	实体类型
传统 NER	人名、地名、机构名、时间
医疗	疾病、药品、症状、手术
金融	股票、指标、机构、事件
电商	品牌、型号、规格、属性
法律	法条、罪名、当事人、证据

关键点：定义标签 → 人工标注 → 训练模型，三步不变，类型随业务自由替换。

自定义类型示例（医疗场景）

输入	患者	服用	阿司匹林	治疗	冠心病
标签	O	O	B-DRUG	O	B-DIS

- "阿司匹林"被识别为药品（DRUG）
- "冠心病"被识别为疾病（DIS）
- 这与识别"马云/PER"或"北京/LOC"在方法上完全一致

迁移性

同一套 BERT+CRF 框架，换标注数据即可迁移到不同领域。

这意味着：

- 不需要为每个新领域开发全新的算法
- 只需要准备领域相关的标注数据
- 模型架构、训练流程、评估方法都是通用的

3.3 NER 的业务应用场景

医疗健康

- **疾病、药品、症状识别**：自动从电子病历中提取关键医疗实体
- **电子病历结构化**：将非结构化的病历文本转化为结构化数据
- **药物副作用监测**：从不良反应报告中自动提取药物和副作用信息

金融投资

- **财报信息抽取**：从年报、季报中自动提取关键财务数据
- **股票/指数/机构识别**：识别新闻中涉及的金融实体
- **金融事件检测**：检测并购、上市、违规等金融事件

法律合规

- **法条引用识别**：从法律文书中自动识别引用的法条
- **当事人/罪名抽取**：从判决书中提取原告、被告、罪名等
- **合同关键条款定位**：自动定位合同中的关键条款

智能电商

- **商品属性结构化**：从商品描述中提取品牌、型号、规格等
- **品牌/型号/规格抽取**：自动构建商品知识图谱
- **用户评价挖掘**：从评论中提取用户关注的属性和情感

新闻媒体

- **事件主体识别**：识别新闻报道中的核心实体
- **人物关系抽取**：从文本中提取人物之间的关系
- **热点话题归因**：追踪热点事件的关键实体

智能客服

- **用户意图实体识别**：识别用户查询中的关键实体（产品名、订单号等）
- **订单号/产品名识别**：自动提取用户提到的订单信息
- **FAQ 精准匹配**：通过实体识别提高问题匹配的精度

3.4 标注体系设计要点

标注方案选择

方案	优点	缺点
BIO	简洁，标签少	单字实体与多字实体 标签混淆
BIOES	边界明确，单字有 S	标签数翻倍，标注更复杂
IO	最简单（2类）	无法区分相邻同类实体

选择建议：

- 初期使用 BIO，数据充足后再考虑 BIOES
- IO 方案仅适用于非常简单的场景，实际中很少使用

嵌套实体处理

标准 BIO 无法表示**嵌套实体**（即一个实体包含在另一个实体内部，如“北京大学”中“北京”是地名，“北京大学”是机构名，二者嵌套）。例如：“北京大学”中：

- 外层是 ORG（机构），包含整个“北京大学”
- 内层也含 LOC（地点），即“北京”

文本：	北	京	大	学
外层：	B-ORG	I-ORG	I-ORG	I-ORG
内层：	B-LOC	I-LOC		

应对方案：

1. **多层标注**：每层独立做 BIO 序列，分别识别不同层级的实体
2. **Span-based 方法**（直接枚举文本中所有可能的连续片段（*span*），对每个 *span* 独立分类是否为实体，不依赖 BIO 标签体系）：天然支持嵌套
3. **忽略处理**：如果业务中嵌套实体比例很低（通常 <5%），可以忽略嵌套，只标注最外层实体

设计原则

1. **类别数不宜过多**：超过 20 个类别会显著增加模型学习难度，标注一致性也难以保证
2. **类别间边界要清晰**：标注歧义越少越好，避免“似是而非”的类别定义
3. **先用 BIO，数据充足后再考虑 BIOES**：BIO 上手快，BIOES 在数据量足够时可以获得更好的边界识别
4. **保持标注一致性**：采用双人标注 + 仲裁机制，确保标注标准的统一

标注一致性保障

标注一致性是 NER 项目成功的关键因素之一：

- **标注指南**：编写详细的标注指南，包含边界案例的处理规则
- **双人标注**：每条数据由两人独立标注
- **仲裁机制**：标注不一致时由第三人裁决
- **Cohen's Kappa**（一种统计指标，衡量两个标注者之间的一致性程度。取值范围 -1 到 1，>0.8 表示一致性很好）：Kappa > 0.8 才表示标注质量可接受

第四章 CRF 与维特比解码

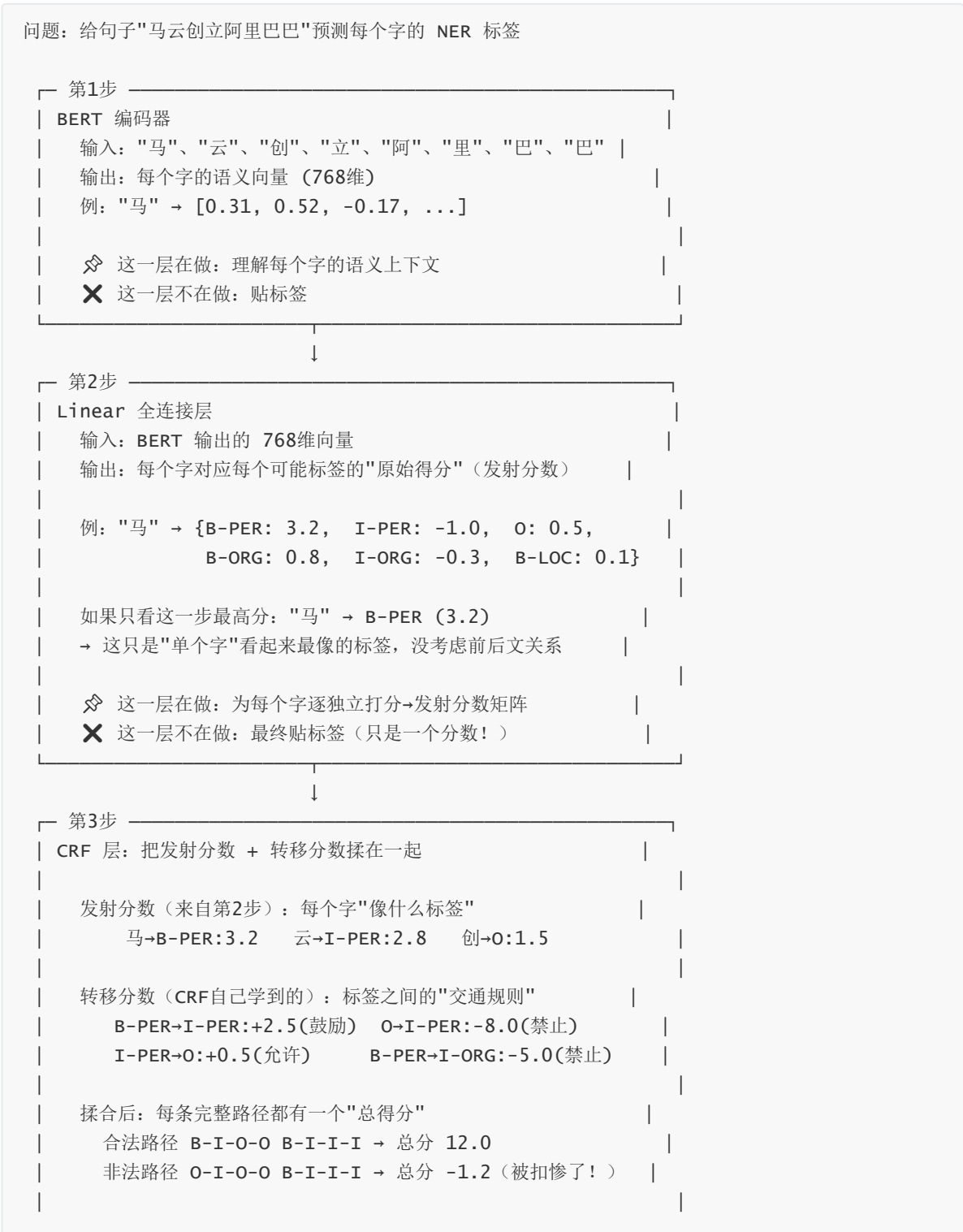
条件随机场（CRF）是序列标注模型中关键的输出层组件，用于建模标签之间的全局依赖关系。维特比（Viterbi）算法则是 CRF 推理阶段寻找最优标签序列的核心算法。

CRF（Conditional Random Field，条件随机场）是一种判别式概率图模型，专门用于对序列数据进行标注。与朴素地逐位置独立预测不同，CRF 考虑了相邻标签之间的依赖关系，能够对整条标签序列进行全局最优求解。

🌀 **新手学习建议：**本章是全文**最难但最重要**的部分。如果你第一次看数学公式感到头晕，请先读每个小节的**"新手直觉类比"**和**"分步数值示例"**，再用数学公式印证你的理解。**先直观后抽象，效果最好。**

本章导航：用一张图理解 CRF + Viterbi 在做什么

标注到底发生在哪一步？这是新手最常困惑的问题。下面这张图把全流程拆成 5 步，**明确标出每一步是"在打分"还是"在出标签"**：



- ✧ 这一层在做：为整条标签序列打出"综合评分"
- ✗ 这一层不在做：还没选出哪条路径最好！只是给每条路径算好了总得分

第4步

viterbi 解码：在所有可能的标签序列中，找到总分最高的那一条

暴力算需要 $3^8 = 6561$ 种路径 → 维特比只用 $8 \times 3^2 = 72$ 次

最优路径：B-PER → I-PER → O → O → B-ORG → I-ORG → I-ORG → I-ORG

总分：12.0 ✓ 最高

- ✧ 这一层在做：在所有候选中找出全局最优的标签序列

第5步

▣ 标注输出 ▣

马	云	创	立	阿	里	巴	巴	
B-PER	I-PER	O	O	B-ORG	I-ORG	I-ORG	I-ORG	

解码结果：马云/PER + 阿里巴巴/ORG

- ✧ 标注就是这一步！整个流程走完，这一行标签序列就是最终的序列标注输出

一句话记忆：BERT 负责"理解"，Linear 负责"打分"，CRF 负责"纠偏"，**Viterbi 负责最终"选标签"**——标注就发生在 Viterbi 解码完成的那一刻。前 3 步全是在为"选标签"做准备，不是在做标注！

🌀 **面试常问**："CRF 层是在做标注吗？" → **不是**！CRF 层只负责计算每条路径的得分，真正的标注是 Viterbi 解码从所有路径中选出最优的那一条。你可以把 CRF 理解为"阅卷老师"（给每张答卷打分），Viterbi 才是"录取官"（按分数决定选谁）。

🔗 深入追踪：一个 "B-PER" 标签是怎么做出来的？

用上面"马云创立阿里巴巴"中**第一个字"马"**最终被标为 B-PER 为例，从头到尾走一遍。

第 1 步：把汉字变成数字——BERT 编码

计算机不认识汉字"马"，只认识数字。BERT 做的就是这件事：

输入："马"

↓ BERT 的 Token Embedding（词嵌入表查表）

[0.31, -0.52, 0.17, 0.89, -0.03, ..., -0.21] ← 768 个数字组成的向量

这个 768 维向量编码了"马"的语义信息——包括它常和"云""克""化"等人名字一起出现、常在句子开头作为姓氏、上下文特征等。

关键点：BERT **不是只看"马"这个字本身**，它通过自注意力机制看见了整个句子"马云创立阿里巴巴"，所以向量里还融入了"马"后面跟着"云"（也像人名）、再后面跟着"创立"（动词，暗示前面可能是主语/人名）这些上下文信息。

第 2 步：把 768 维向量压成标签分数——Linear 层

BERT 的输出是 768 个浮点数，但我们需要的是**每个标签的得分**。Linear 层就是做这个映射的：

"马"的 BERT 向量：[0.31, -0.52, 0.17, 0.89, ..., -0.21] （768维）

↓ Linear 层做矩阵乘法：向量 $\times$ 权重矩阵 W + 偏置 b
| W 的大小：(768, num_labels)，假设 6 个标签
| b 的长度：6

结果为 6 个发射分数：

标签	得分	
B-PER	3.2	← 最高！"马"最像人名开头
I-PER	-1.0	"马"不太像人名内部
B-ORG	0.8	"马"有点可能也是机构名开头
I-ORG	-0.3	不太像
B-LOC	0.1	略有一点地名开头的可能
O	0.5	也像普通字

B-PER 的 3.2 分是怎么来的？

本质是一次**加权求和**：

$$\begin{aligned}\text{score}(\text{B-PER}) &= (\text{向量第1维} \times W[1][\text{B-PER}]) + (\text{向量第2维} \times W[2][\text{B-PER}]) + \dots \\ &\quad + (\text{向量第768维} \times W[768][\text{B-PER}]) + \text{偏置}[\text{B-PER}] \\ &= 3.2\end{aligned}$$

- 权重矩阵 W 是**训练学出来的**——训练数据里有大量"马云/B-PER"这样的标注，训练过程中反向传播不断调整 W ，使得像"马"这样的人名首字在第 B-PER 位置上的得分越来越高
- BERT 输出的向量里那些代表人名特征的维度会被分配较大的正权重，反之被分配负权重

⚠ **此时如果粗暴地选最高分 (3.2)，"马"就已经被标为 B-PER 了。**但这是"孤立法"——只看了"马"这一个字，没考虑它前后标签的合法性。如果前面已经是 I-PER 了，那"马"再出 B-PER 就是非法的（I 后面不能直接以另一个 B 开头）。所以需要下一步。

第 3 步：引入"连带责任"——CRF 转移矩阵

CRF 手里有一张**转移矩阵**，它规定了标签之间谁能接谁、得分多少：

训练后学到的转移矩阵（部分）：

从 \ 到	B-PER	I-PER	B-ORG	O	...	
<START>	+0.5	-3.0	+0.5	+0.3		← 句首：B-PER和B-ORG和O都合法
B-PER	-1.0	+2.5	-5.0	+0.8		← B-PER后：鼓励I-PER，允许O，禁止B-ORG
I-PER	-3.0	+1.8	-3.0	+0.5		← I-PER后：继续I-PER或结束为O
O	+1.0	-8.0	+1.0	+0.3		← O后：可以开始B-PER/B-ORG，禁止直接出I
...						

对于"马"这个位置（t=0，句子开头），它的前驱是 <START>（特殊起始标签）：

"马"标 B-PER 的总得分 = 发射(B-PER) + 转移(START→B-PER)
= 3.2 + 0.5
= 3.7 ✓

"马"标 I-PER 的总得分 = -1.0 + (-3.0)
= -4.0 ✗ 被转移分扣惨了（句首不能是I！）

"马"标 O 的总得分 = 0.5 + 0.3
= 0.8 △ 合法但不高

第 4 步：全局最优——Viterbi 拍板

现在每个字的位置上，每个标签候选都有一个累计到该位置的最优得分。维特比不是只看"马"这一个字——它要看完整条序列：

每个时间步每个标签候选的 dp 得分（向前递推，保留最优路径）：

t=0 (马): dp[B-PER]=3.7 ✓ dp[I-PER]=-4.0 dp[O]=0.8
t=1 (云): dp[B-PER]=5.0 dp[I-PER]=9.0 ✓ dp[O]=5.2
 ↑ 来自 B-PER ↑ 来自 B-PER ↑ 来自 B-PER
t=2 (创): dp[B-PER]=10.2 dp[I-PER]=11.4 dp[O]=9.5
 ↑ 来自 I-PER ↑ 来自 I-PER ✓ ↑ 来自 I-PER
t=3 (立): dp[B-PER]=11.6 dp[I-PER]=13.0 dp[O]=11.9 ✓
 ↑ 来自 O ↑ 来自 O ↑ 来自 I-PER
... (后续同理递推)

最终回溯时：

t=7 (巴) → 标签 O，前驱 O
t=6 (巴) → 标签 I-ORG，前驱 I-ORG
...
t=1 (云) → 标签 I-PER，前驱 B-PER
t=0 (马) → 标签 B-PER，前驱 START
 ↑
 这是全局路径的一部分，不是"马"独自选的！

总结：一个 B-PER 的诞生

"马"这个汉字

↓

▼ BERT (理解语义 + 上下文)

[0.31, -0.52, ...] 768维向量

↓

▼ Linear (独立打分)

B-PER: 3.2分 ← "马"本身最像人名开头

↓

▼ + CRF 转移约束 (前驱是 START, START→B-PER 合法)

最终路径得分贡献: $3.2 + 0.5 = 3.7$

↓

▼ + Viterbi (看整句最优路径)

↓ 往前看: 云→I-PER (合理延续) → 创→O (人名结束) → ...

↓ 回头看: 前驱是 START (句首合法)

↓ 综合判断: 选 B-PER 作为 t=0 的最优标签

↓

🏁 B-PER

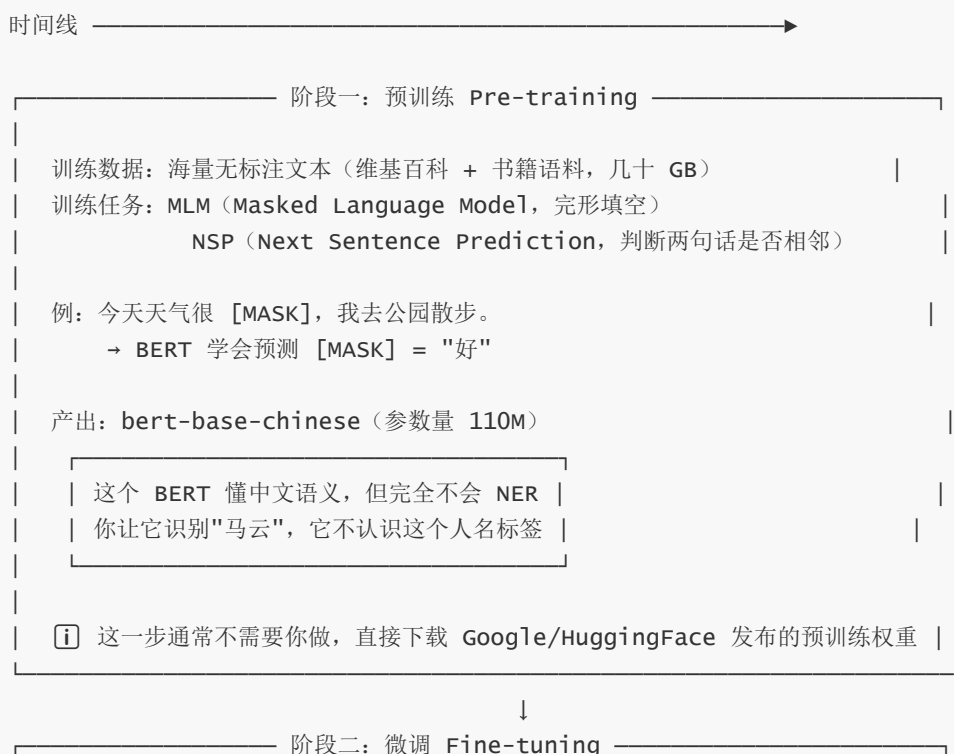
一句话: B-PER 不是"马"这个字自己决定的, 它是由 **BERT 编码 (理解上下文) + Linear 打分 (逐个评估) + CRF 约束 (禁止非法转移) + Viterbi 全局选优 (统筹全句)** 四者联合决定的。去掉任何一环都会影响最终结果。

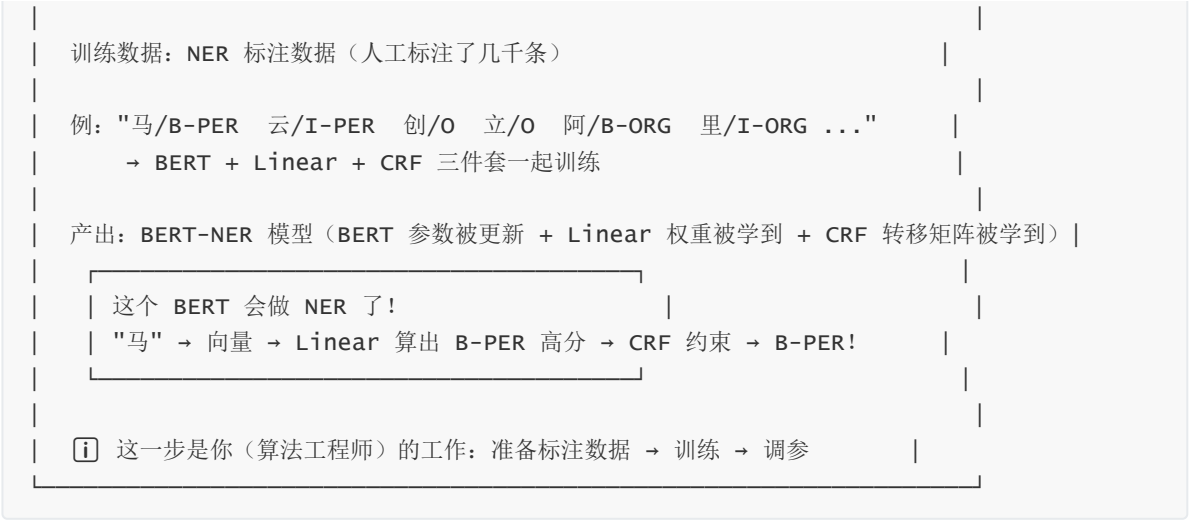
⚠️ 关键澄清: 这里的 BERT 是"训练完成的"吗?

这是一个新手极易混淆的核心问题。答案是: **是, 但必须区分"两次训练"**。

🎯 **新手类比:** BERT 就像一个人。第一阶段 (预训练) 是"上学读书"——读了海量书籍 (维基百科、新闻、小说等), 学会了识字、语法和常识。第二阶段 (微调) 是"入职培训"——被招进 NER 项目组后, 用标注数据 ("马云=B-PER, 云=I-PER, ...") 专门训练他认人名识地名。我们用的是经过入职培训的 BERT, 不是那个刚毕业啥都懂但不会做 NER 的学生。

BERT 的两个阶段 (一张表看懂)



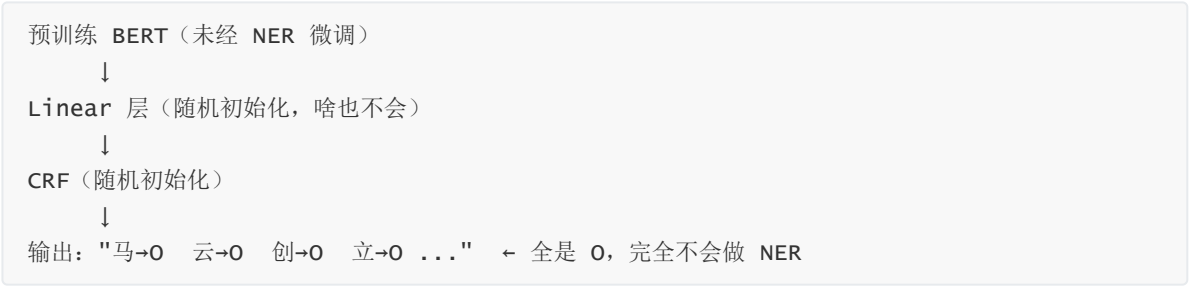


哪些参数在微调中被更新？

组件	微调前	微调后	谁更新的？
BERT 内部权重 （12层 Transformer，110M参数）	Google 预训练的值	被更新 （略有调整，适应 NER任务）	NER 标注数据的反向传播
Linear 层权重 （768 × num_labels 矩阵）	随机初始化	被学出来（如 B-PER 维度的正权重变大）	NER 标注数据的反向传播
CRF 转移矩阵 （num_labels × num_labels）	随机初始化	被学出来（如 O→I-PER = -8.0）	NER 标注数据的反向传播

三个组件是联合训练（end-to-end）的：同一个损失函数、同一次反向传播，BERT 参数、Linear 权重、CRF 转移矩阵同时更新。不是先训好一个再训下一个。

如果不微调，直接用预训练 BERT 做 NER？



因为：

- BERT 虽然懂"马"、"云"这些字的语义，但它不知道"B-PER"是什么意思——预训练阶段根本没学过这个标签
- Linear 层是随机初始化的，无论 BERT 输出什么向量，乘上去都是随机乱猜
- CRF 转移矩阵也是随机的，不懂标签间的约束关系

微调就是教会这三者协同工作的过程。

一句话总结

问题	答案
此处用的是训练完成的 BERT 吗？	是——但经过了 两次训练 ：预训练（学中文）→ 微调（学 NER）
预训练 BERT 能直接做 NER 吗？	不能 ——它不懂 B-PER/I-PER 这些标签
Linear 和 CRF 是预训练好的吗？	不是 ——它们随机初始化，完全靠 NER 标注数据学出来
三者怎么训练的？	联合训练（end-to-end） ，一次反向传播同时更新三者

4.1 为什么需要 CRF？

🌀 新手直觉类比：GPS 导航 vs 逐路口猜方向

想象你在北京开车去一个陌生地点。你有两种方式：

方式 A（不用 CRF = 逐 token Softmax）：每个路口独立猜方向——左转？直行？右转？完全不管上一个路口怎么走的。结果可能走出"左转-右转-左转-掉头"这种疯路线，甚至开到死胡同里（相当于预测出 `O → I-PER` 这种非法序列）。

方式 B（用 CRF）：像 GPS 导航一样，**通盘考虑整条路径**，知道"这条路接那条路才合理"，最终选出一条从起点到终点的最优路线。

CRF 就是序列标注模型的"导航大脑"——它不会让每个字孤立地选标签，而是统筹整句话找到最合理的标签序列。

独立预测（Softmax per token）的缺陷

如果不使用 CRF，直接对每个 token 的标签做 **Softmax**（一种将实数向量归一化为概率分布的函数，常用于多分类任务的输出层）预测，会出现严重问题：

每个 token 的标签独立预测，不考虑相邻标签之间的约束。

这会导致非法标签转移，例如：

预测输出：B-PER I-LOC I-ORG B-ORG
AAAA A
非法：I-LOC 不能跟在 B-PER 后（类型不一致）
非法：I-ORG 前面没有 B-ORG（I 不能独立出现）

常见非法转移模式

非法模式	原因
I-X 出现在序列起始位置	I 标签必须跟在 B-X 或 I-X 之后
I-X 紧接在 B-Y (X≠Y) 之后	实体类型不一致
I-X 紧接在 O 标签之后	I 标签不能在非实体之后开始

CRF 的解决方案

CRF 在输出层加入**全局转移约束**，对整条标签序列联合建模，找全局最优输出：

$$P(Y|X) \propto \exp(\sum_t \text{emission}(y_t, x_t) + \sum_t \text{transition}(y_{t-1}, y_t))$$

- **emission**（发射分数）：当前 token 属于某标签的得分（来自编码器的输出）
- **transition**（转移分数）：相邻标签对的转移得分（可学习参数）

CRF 通过联合对整条序列打分，取全局最优，从而避免非法转移。

4.2 CRF 原理

总分计算

CRF 对一条标签序列 Y 的打分公式为：

$$\text{score}(Y, X) = \sum_t \text{emission}(y_t, x_t) + \sum_t \text{transition}(y_{t-1}, y_t)$$

整个得分由两部分组成：**发射分数**和**转移分数**。

发射分数（Emission Score）—— "这个字本身像什么标签"

发射分数来自底层编码器（LSTM / BERT）的 token 表示，经过 **Linear 层**（全连接层，将高维向量映射到标签空间）映射到每个标签的得分。

```
# BERT + Linear 输出
logits = linear(bert_out) # (T, num_tags)
# logits[t][j] = emit(token_t → tag_j)
```

- `logits[t][j]` 表示第 t 个 token 属于第 j 个标签的局部得分
- 这部分得分**不考虑序列约束**，是独立于上下文标签的

🔍 分步数值示例 — 以"马云创立阿里巴巴"为例

假设我们只有 3 种标签：**B-PER**（人名开头）、**I-PER**（人名内部）、**O**（非实体）。BERT+Linear 层为每个字输出的发射分数如下：

字	B-PER 得分	I-PER 得分	O 得分	模型"觉得"
马	3.2	-1.0	0.5	"马"很可能是人名开头
云	0.3	2.8	0.1	"云"很可能是人名内部
创	-0.5	-1.2	1.5	"创"不太像人名，更像普通字
立	-0.8	-1.5	1.2	"立"也不太像人名

如果只看发射分数，逐位置选最高分 → **B-PER, I-PER, O, O**，恰好正确！但**这是运气好**，实际中发射分数容易出错，尤其是歧义字。CRF 的转移分数就是来"纠偏"的。

【深入理解】"发射"这个名称从哪来？

发射分数 (Emission Score) 这个名字来源于**隐马尔可夫模型 (HMM)** 的术语。在 HMM 中，隐藏状态 (标签) 会"发射"出观测值 (token)，发射概率 $P(token|tag)$ 表示"给定标签，生成这个词的概率"。

CRF 借用了这个术语，但方向反了——CRF 是判别式模型，发射分数表示的是"给定 token，它属于某标签的得分"。**名字一样，意思不同，新手别搞混。**

转移分数 (Transition Score) —— "两个标签相邻合不合理"

转移分数是一个可学习的矩阵 T ，大小为 `(num_tags, num_tags)`：

- $T[i][j]$ 表示从标签 i 转移到标签 j 的得分
- 正值表示该转移是被鼓励的，负值表示该转移是被抑制的

完整的 3 标签转移矩阵示例：

从 \ 到	B-PER	I-PER	O
B-PER	1.2	2.5	0.8
I-PER	-3.0	1.8	0.5
O	1.0	-8.0	0.3

解读这张转移矩阵：

转移	得分	含义
<code>B-PER → I-PER</code>	+2.5 (高)	人名开头 → 人名内部，合理的实体延续， 鼓励
<code>I-PER → I-PER</code>	+1.8 (高)	人名内部继续，长实体支持的， 鼓励
<code>O → I-PER</code>	-8.0 (极低)	非实体后面突然冒出"人名内部"？ 严重违规，禁止！
<code>I-PER → B-PER</code>	-3.0 (低)	人名内部后又来一个人名开头，虽然语法上可能存在，但罕见， 抑制

发射 + 转移 = 完整路径得分 (具体数值计算)

把发射分数和转移分数结合起来，我们计算一条标签序列的总分：

示例序列："马 云 创 立" → 预测标签 "B-PER I-PER O O"			
总分计算（一步步加）：			
位置1，马→B-PER：	发射 = 3.2		= 3.2
位置2，云→I-PER：	发射 = 2.8	+ 转移(B-PER→I-PER) = +2.5	= +5.3
位置3，创→O：	发射 = 1.5	+ 转移(I-PER→O) = +0.5	= +2.0
位置4，立→O：	发射 = 1.2	+ 转移(O→O) = +0.3	= +1.5
总分 = 12.0			← 合法路径，得分较高

再来算一条非法路径作对比：

"马 云 创 立" → 预测标签 "O I-PER O O" （注意：O后面直接I-PER，非法！）

位置1, 马→O:	发射 = 0.5	= 0.5
位置2, 云→I-PER:	发射 = 2.8 + 转移(O→I-PER) = -8.0	= -5.2 ← 扣分严重!
位置3, 创→O:	发射 = 1.5 + 转移(I-PER→O) = +0.5	= +2.0
位置4, 立→O:	发射 = 1.2 + 转移(O→O) = +0.3	= +1.5
总分 = -1.2		← 非法路径，得分极低!

结论一目了然：合法路径总分 **12.0**，非法路径总分 **-1.2**。CRF 选择得分最高的路径——自动就避开了非法序列。这就是 CRF 的核心魅力！

训练如何学习转移约束

训练时通过最大化正确标签序列的得分，转移矩阵会自动学习到：

- 合法转移 (如 B-PER → I-PER) → 高分
- 非法转移 (如 O → I-PER) → 极大负数

这意味着**不需要手动设定约束规则**，CRF 会从数据中自动学习标签转移的合法性。

4.3 维特比解码 (Viterbi Decoding)

核心理念

维特比解码使用**动态规划** (Dynamic Programming, 一种将复杂问题分解为重叠子问题并递推求解的算法思想，通过存储中间结果避免重复计算) 方法，在所有合法标签序列中找到使得总得分最大的路径。

- 暴力枚举所有路径的复杂度为 $O(K^T)$ (指数级，不可行)
- 维特比算法将其降至 $O(T \times K^2)$ (多项式级，实际可用)

其中 T 是序列长度， K 是标签数。

🧠 **新手直觉类比：**维特比算法就像你在迷宫里找最短路径。不是把所有路都走一遍（指数级），而是边走边记——到每个路口的“最佳走法”，最后从终点顺着“最佳路线”往回走就行。

🔧 手把手数值示例：一句话跑通 Viterbi

问题：序列"马云创"，只有 3 个标签 {B, I, O} (简化版)，用已学到的发射分数和转移矩阵，找到最优标签序列。

已知数据：

发射分数矩阵 (3个字 × 3个标签)：

字	B	I	O
马	3.2	-1.0	0.5
云	0.3	2.8	0.1
创	-0.5	-1.2	1.5

转移矩阵 (3×3)：

从\到	B	I	O
B	1.2	2.5	0.8
I	-3.0	1.8	0.5
O	1.0	-8.0	0.3

第一步：初始化 (t=0, "马")

第一个字没有"上一个标签", 所以转移分数为 0, dp 就是发射分数:

```
dp[0][B] = emit(马→B) = 3.2
dp[0][I] = emit(马→I) = -1.0
dp[0][O] = emit(马→O) = 0.5
```

当前最优: 马 → B (3.2)

第二步：递推 (t=1, "云")

现在"云"有 3 个候选标签, 每个候选需要看从 t=0 的 3 个标签转移过来哪个得分最高:

计算 dp[1][B] ("云"标 B) :

```
方案1: 从 B 转来 → dp[0][B] + trans[B→B] + emit(云→B) = 3.2 + 1.2 + 0.3 = 4.7
方案2: 从 I 转来 → dp[0][I] + trans[I→B] + emit(云→B) = -1.0 + (-3.0) + 0.3 = -3.7
方案3: 从 O 转来 → dp[0][O] + trans[O→B] + emit(云→B) = 0.5 + 1.0 + 0.3 = 1.8
```

→ 最大值 = 4.7, 来自标签 B (即 马→B 后接 云→B)

计算 dp[1][I] ("云"标 I) :

```
方案1: 从 B 转来 → 3.2 + 2.5 + 2.8 = 8.5 ← 最高!
方案2: 从 I 转来 → -1.0 + 1.8 + 2.8 = 3.6
方案3: 从 O 转来 → 0.5 + (-8.0) + 2.8 = -4.7 ← 被 -8.0 拖累
```

→ 最大值 = 8.5, 来自标签 B (即 马→B 后接 云→I)

计算 dp[1][O] ("云"标 O) :

```
方案1: 从 B 转来 → 3.2 + 0.8 + 0.1 = 4.1
方案2: 从 I 转来 → -1.0 + 0.5 + 0.1 = -0.4
方案3: 从 O 转来 → 0.5 + 0.3 + 0.1 = 0.9
```

→ 最大值 = 4.1, 来自标签 B

路径	得分	前驱
dp[1][B]	4.7	← B
dp[1][I]	8.5	← B
dp[1][O]	4.1	← B

第三步：递推 (t=2, "创")

计算 $dp[2][B]$ ("创"标 B) :

方案1: 从 B 转来 $\rightarrow 4.7 + 1.2 + (-0.5) = 5.4$
方案2: 从 I 转来 $\rightarrow 8.5 + (-3.0) + (-0.5) = 5.0$
方案3: 从 O 转来 $\rightarrow 4.1 + 1.0 + (-0.5) = 4.6$

$\rightarrow$ 最大值 = **5.4**, 前驱 = **B**

计算 $dp[2][I]$ ("创"标 I) :

方案1: 从 B 转来 $\rightarrow 4.7 + 2.5 + (-1.2) = 6.0$
方案2: 从 I 转来 $\rightarrow 8.5 + 1.8 + (-1.2) = 9.1 \leftarrow$ 最高!
方案3: 从 O 转来 $\rightarrow 4.1 + (-8.0) + (-1.2) = -5.1$

$\rightarrow$ 最大值 = **9.1**, 前驱 = **I**

计算 $dp[2][O]$ ("创"标 O) :

方案1: 从 B 转来 $\rightarrow 4.7 + 0.8 + 1.5 = 7.0 \leftarrow$ 最高!
方案2: 从 I 转来 $\rightarrow 8.5 + 0.5 + 1.5 = 10.5 \leftarrow$ 才是最高!!!
方案3: 从 O 转来 $\rightarrow 4.1 + 0.3 + 1.5 = 5.9$

$\rightarrow$ 最大值 = **10.5**, 前驱 = **I**

路径	得分	前驱
$dp[2][B]$	5.4	$\leftarrow B$
$dp[2][I]$	9.1	$\leftarrow I$
$dp[2][O]$	10.5	$\leftarrow I$

第四步：终止 (找终点)

$best_last = \max(dp[2][B]=5.4, dp[2][I]=9.1, dp[2][O]=10.5)$
 $= O (10.5)$

第五步：回溯

t=2 (创) $\rightarrow$ 标签 O, 前驱 I
t=1 (云) $\rightarrow$ 标签 I, 前驱 B
t=0 (马) $\rightarrow$ 标签 B

最终路径: B $\rightarrow$ I $\rightarrow$ O (马=B, 云=I, 创=O)

验证：这条路径的总分是多少？

马→B:	emit = 3.2		= 3.2
云→I:	emit = 2.8	+ trans(B→I) = +2.5	= +5.3
创→O:	emit = 1.5	+ trans(I→O) = +0.5	= +2.0
总和 = 10.5 ✓ (与 dp[2][0] 一致!)			

最终结果解读： B-I-O 表示"马"是人名开头、"云"是人名内部、"创"是非实体——恰好是 马/云/B-PER 云/I-PER 创/O，符合我们的预期！

算法步骤（公式总结）

第一步：初始化

$$dp[0][j] = emit[0][j]$$

对于序列的第一个位置，直接使用发射分数初始化。

第二步：递推

$$dp[t][j] = \max_i (dp[t-1][i] + trans[i][j]) + emit[t][j]$$

对于每个时间步 t 和每个标签 j ，找从前一时间步所有标签 i 转移过来的最大得分，加上当前发射分数。

第三步：记录路径

$$ptr[t][j] = \arg \max_i (dp[t-1][i] + trans[i][j])$$

记录每个位置的最优前驱标签，用于最终回溯。

第四步：终止

$$best_last = \arg \max_j (dp[T-1][j])$$

在最后一个时间步，选择得分最高的标签作为终点。

第五步：回溯

从 `best_last` 沿 `ptr` 反向追踪，得到完整的标签序列。

路径示意（3 标签 × 4 时间步）

时间步：	t=0	t=1	t=2	t=3
标签 B:	B →	B →	B →	B
标签 I:	I →	I →	I →	I
标签 O:	O →	O →	O →	O
最优路径：	B →	I →	I →	O

复杂度分析

操作	复杂度
递推	$O(T \times K^2)$
回溯	$O(T)$
总计	$O(T \times K^2)$

对比暴力枚举的 $O(K^T)$ ，维特比算法在 $K = 10, T = 50$ 的典型场景下，将计算量从 10^{50} 降至 5000。

4.4 训练与推理

训练阶段

目标：最大化正确标签序列的对数概率

$$\log P(Y^*|X) = \text{score}(Y^*, X) - \log Z(X)$$

其中：

- Y^* 是正确的标签序列
- $Z(X) = \sum_Y \text{score}(Y, X)$ 是所有可能序列得分之和，称为**配分函数** (Partition Function，在概率模型中对所有可能状态的得分求和，用于将得分归一化为概率)
- $Z(X)$ 通过**前向算法** (Forward Algorithm，一种动态规划算法，以 $O(T \times K^2)$ 复杂度高效计算配分函数，避免枚举所有路径) 高效计算

训练过程使用梯度下降优化参数（包括转移矩阵 T 和编码器参数），使得正确序列的概率最大化。

推理阶段

目标：找到得分最高的标签序列

$$Y^* = \arg \max_Y \text{score}(Y, X) = \text{Viterbi}(\text{emission}, \text{transition})$$

推理时不需要计算 $Z(X)$ ，直接使用维特比解码找最优路径即可。

复杂度： $O(T \times K^2)$

CRF 并非必选项

LSTM 时代：CRF 收益显著。LSTM 的**感受野** (Receptive Field，模型能“看到”的输入范围。LSTM 主要依赖单向历史信息，对全局上下文建模能力有限) 难以学好标签依赖，CRF 弥补了这一短板。

BERT 时代：

- BERT 的双向注意力已能捕捉全局标签依赖
- CRF 带来的 F1 提升通常 < **0.5%**
- 维特比解码增加**推理延迟** (Inference Latency，模型做出一次预测所需的时间)，因为它是串行计算，**批处理** (Batch Processing，同时处理多个样本以提高吞吐量) 受限
- 轻量部署/实时场景：直接 Softmax 更实用

经验结论：

BERT 基座 + Softmax 直接可用；若数据少或标签约束复杂，再加 CRF。

实践建议

场景	建议
BERT + 大量标注数据	Softmax 足够，CRF 提升有限
BERT + 少量数据	加 CRF，利用转移约束减少非法预测
LSTM 基座	强烈建议加 CRF
实时/低延迟场景	优先 Softmax（避免维特比串行开销）
标签约束复杂	加 CRF（自动学习约束）

第五章 评估方式与常见问题

NER 模型的评估是项目落地的关键环节，错误的评估方式会导致对模型性能的误判。

5.1 为什么 Token 准确率没有意义？

根本问题

NER 的业务价值在于能否完整抽取出实体，不在于 token 级别的对与错。

常见误操作：用 **Token 准确率**（Token Accuracy，逐 token 比对预测是否正确然后求平均）衡量 NER 模型性能。

新手类比：Token 准确率就像用"每个字写对了没"来判断一篇文章的翻译质量——哪怕整句话的意思完全错乱，只要大部分字碰巧一样，得分还很高。NER 也一样，实体有没有抽对才是关键。

为什么准确率虚高？

大多数 token 的标签是 **O**（非实体），**O** 标签天然占主导地位（通常 80%-90% 的 token 是 O）。

极端示例 1：全预测 O 的"好成绩"：

句子：北京 是 中国 的 首都

真实：B-LOC O B-LOC O O

预测：O O O O O （全预测O）

Token 准确率 = 3/5 = 60%

=> 模型没有识别任何实体！但准确率仍有 60%！

极端情况下：一段文本中实体只占 10% 的 token，**全预测 O 的准确率仍高达 90%！**

示例 2：边界错的"高分幻觉"：

句子：我 去 北 京 天 安 门 玩
真实：O O B-LOC I-LOC I-LOC I-LOC I-LOC O
预测：O O B-LOC I-LOC O O O O （只抽了"北京"）

Token 准确率 = 6/8 = 75%
实际：漏掉了"天安门"这个完整实体！75% 的准确率完全掩盖了问题！

示例 3：三种错误的 Token 准确率对比（同一句话）：

真值：B-PER I-PER O B-ORG I-ORG I-ORG （马云 创立 阿里巴巴）

错误类型	预测	Token 准确率	实际 F1	问题
漏识	O O O B-ORG I-ORG I-ORG ORG	83%	67%	"马云"完全没识别，准确率却很高
边界错 + 类型错	B-PER I-PER O B-PER I-ORG I-ORG	83%	50%	"阿里巴巴"抽成 PER + 边界乱，准确率仍 83%
全部预测 O	O O O O O O	50%	0%	啥都没抽到，还有 50%！（因为有3个O猜对）

结论：Token 准确率对于 NER 是完全失真的指标，它无法区分"差模型"和"好模型"。请务必使用实体级 F1。

Token 准确率掩盖的错误类型

错误类型	描述	Token 准确率的表现
边界截断	抽到"北京天安"（少了"门"）	大部分 token 仍算对，准确率高
漏识	一个实体完全没被识别	O 标签占多数，准确率影响小
类型错误	PER 预测为 ORG	边界正确时部分 token 仍可能算对

这些错误在 Token 准确率下均被掩盖，因此 Token 准确率对 NER 任务来说是一个误导性指标。

5.2 正确的评估方式：实体级 F1

评估粒度

一个实体被视为正确，当且仅当实体的起止边界与类型均完全匹配（Exact Match，精确匹配）。

即：只看完整的实体 span 是否被正确识别，而不是逐 token 比对。

三个核心指标

Precision（精确率）

P = 正确识别实体数 / 模型预测总实体数

含义：在模型预测的所有实体中，有多少是真正正确的。衡量模型"不误报"的能力。

Recall（召回率）

R = 正确识别实体数 / 标注集真实实体总数

含义：在所有真实实体中，有多少被模型正确识别。衡量模型"不遗漏"的能力。

F1 分数

F1 = 2 * P * R / (P + R)

F1 是 P 和 R 的调和平均（Harmonic Mean，与算术平均不同，调和平均对较小的值更敏感，因此 F1 只有在 P 和 R 都高时才能得到高分，避免了一个指标高一个指标低时的虚高），兼顾精确率与召回率，是 NER 首选指标。

🔗 分步练习：三个递进难度的 F1 计算示例

在开始之前，先理解关键概念：

🧠 新手类比：NER 评估就像警察抓通缉犯。Precision = 你抓的人里真正有罪的有几个（不冤枉好人），Recall = 所有罪犯你抓到了几个（不放过坏人），F1 = 两者的平衡分。如果一个警察抓了 100 人但只有 1 个真罪犯（Precision=1%），或者 10 个罪犯只抓到 1 个（Recall=10%），都不是好警察。

实体级"正确"的判定标准：

- ☑ 实体的起止位置和类型都要完全匹配
- ✗ 边界差一个字也不算对
- ✗ 实体类型错了也不算对

示例 1（入门）：最简单的场景

句子: "马云创立阿里巴巴"

真实实体: {马云/PER} {阿里巴巴/ORG}

预测实体: {马云/PER} {阿里巴巴/ORG}

指标	计算	结果
TP（真正例）	完全匹配的有: "马云/PER"、"阿里巴巴/ORG"	2
FP（假正例）	预测了但不对的有: 无	0
FN（假负例）	应该预测但没预测到的: 无	0
Precision	TP / (TP + FP) = 2 / 2	1.00
Recall	TP / (TP + FN) = 2 / 2	1.00
F1	2 * 1 * 1 / (1 + 1)	1.00 🌟

示例 2（进阶）：边界错误

句子: "马云创立阿里巴巴"

真实实体: {马云/PER} {阿里巴巴/ORG}

预测实体: {马云/PER} {阿里/ORG} {巴巴/PER}

☒

☒边界错

☒彻底错

指标	说明	结果
TP	只有"马云/PER"完全正确	1
FP	"阿里/ORG"边界错 + "巴巴/PER"类型错都算 FP	2
FN	"阿里巴巴/ORG"没被正确识别	1
Precision	$1/(1+2) = 33.3\%$	模型预测了 3 个实体，只对了 1 个
Recall	$1/(1+1) = 50\%$	总共 2 个真实实体，找到了 1 个
F1	$2 \times 0.333 \times 0.5 / (0.333 + 0.5)$	40% ⚠

注意：即使实体类型猜对（如"阿里/ORG"），边界错了也按不匹配处理！

示例 3（实战）：含多句的综合性评估

句1: 真实: {北京/LOC} 预测: {北京/LOC} ← TP

句2: 真实: {阿里巴巴/ORG} 预测: {阿里/ORG} {巴巴/ORG} ← FP + FP + FN

句3: 真实: {马云/PER} {淘宝/ORG} 预测: O (什么都没预测) ← FN + FN

句4: 真实: 无实体 预测: {腾讯/ORG} ← FP

拆解统计：

实体	状态	类型
北京/LOC (预=真)	☒ 完全匹配	TP
阿里/ORG (纯预测)	☒ 真实"阿里巴巴"是 4 个字, "阿里"只有 2 个字	FP
巴巴/ORG (纯预测)	☒ 同上, 多余预测	FP
阿里巴巴/ORG (真)	☒ 没有被正确识别出来	FN
马云/PER (真)	☒ 句3 什么都没预测	FN
淘宝/ORG (真)	☒ 句3 什么都没预测	FN
腾讯/ORG (纯预测)	☒ 句4 本无实体的误报	FP

指标	计算	结果
TP		1
FP	3 个	3
FN	3 个	3
Precision	1/4	25%
Recall	1/4	25%
F1	$2 \times 0.25 \times 0.25 / (0.25 + 0.25)$	25% 🎯

这个模型虽然 Token 准确率可能不低，但实体级 F1 只有 25%——严重不及格。

原始计算示例

评估工具

使用 `segeval` 库可以直接对 BIO/BIOES 格式的标签序列计算实体级指标：

```
from segeval.metrics import (
    precision_score, recall_score,
    f1_score, classification_report
)

# y_true 和 y_pred 是标签序列的列表
# 例如: y_true = [['B-PER', 'I-PER', 'O', 'B-ORG', 'I-ORG']]
print(classification_report(y_true, y_pred))
```

安装： `pip install segeval`

5.3 常见错误模式与优化方向

边界截断 / 扩张

项目	说明
现象	应抽"北京天安门" → 抽成"北京天安" (少了"门")
影响	边界识别不准，F1 直接归零（实体级 Exact Match 不通过）
优化	增加长实体训练样本；适当调大序列最大长度

边界错误是 NER 中最常见的错误类型，尤其对长实体影响显著。因为实体级评估要求完全匹配，少一个字或多一个字都算错误。

实体类型混淆

项目	说明
现象	"阿里巴巴"(ORG) 被预测为 PER
影响	边界正确但类型错，实体级评估仍算错误
优化	增加类别混淆样本；优化标签设计，避免歧义类别

类型混淆常见于语义相近的类别之间，如人名(PER)和机构名(ORG) ("阿里巴巴"既可以是公司也可以是人)、地名(LOC)和机构名(ORG) ("北京大学")。

漏识 (低 Recall)

项目	说明
现象	真实实体完全未被识别
影响	召回率下降，通常由训练集覆盖不足导致
优化	扩充标注数据；远程监督补充稀有实体

漏识常见于低频实体、新词、以及训练集中从未出现过的实体类型。

误识 (低 Precision)

项目	说明
现象	非实体被预测为实体
影响	精确率下降，常见于通用词与专业词混淆
优化	负样本补充；加入领域上下文特征

误识常见于"边界模糊"的词，如某些普通名词在特定上下文中可能是实体。

5.4 冷启动与远程监督 (Distant Supervision)

业务落地的核心痛点：缺标注数据

- 人工标注成本高（每人每天约 1000 条）
- 初期质量不稳定、标注歧义大
- 新领域/新类别启动周期长
- 专家标注资源稀缺（医疗、法律等领域需要领域专家）

远程监督 (Distant Supervision)

远程监督是一种利用已有知识资源（如词典、知识库）自动为文本生成标注的方法，无需人工参与。

核心思路：用已有**知识库/词典**（预先收集的结构化数据，如药品名列表、公司名数据库等）匹配文本，自动生成标注，无需人工。

示例：

词典: {'阿司匹林': DRUG, '冠心病': DIS}
文本: 患者服用阿司匹林治疗冠心病
自动生成标注: O O B-DRUG I-DRUG I-DRUG O B-DIS I-DIS I-DIS

远程监督的优势与风险

方面	说明
优势	快速获得大量训练数据，免人工标注
风险 1	词典覆盖率有限，可能漏标（未收录的实体不会被标注）
风险 2	歧义词误标（"苹果"在水果语境中被误标为公司名）
对策	噪声需通过置信度过滤控制

远程监督生成的数据存在噪声，但作为**冷启动**（Cold Start，指系统在没有任何历史数据或标注的情况下从零开始构建模型）的种子数据是可接受的。后续通过模型迭代和人工审核可以逐步提升数据质量。

5.5 滚雪球式迭代优化

核心思想

模型越好，能筛选的高质量数据越多；数据越多，模型越好——形成正向飞轮。

迭代流程

步骤	操作	说明
1. 少量人工标注	100~500 条精标数据	覆盖典型模式
2. 训练初始模型	BERT + CRF	得到初始 NER 模型
3. 预测 + 过滤	对无标注文本预测	保留高置信样本
4. 人工审核扩充	审核高置信结果	修正后加入训练集
5. 重训 → 迭代	扩充数据重训	数据集指数增长

关键控制点

- 置信度阈值**：只将模型打分 > 0.9 的预测结果纳入扩充，抑制噪声积累
- 人工抽检**：每轮扩充前随机抽查 5%~10%，发现质量下降立即暂停
- 验证集独立**：保留一份全人工标注的测试集，作为每轮迭代的客观评估基准

为什么滚雪球有效？

- 初始模型虽然不完美，但在高置信样本上的预测通常是正确的
- 通过置信度筛选，只保留高质量预测，避免噪声积累
- 每轮迭代后数据量增长，模型能力提升，下一轮的高置信样本更多
- 人工审核是质量守门员，确保数据不会因噪声累积而退化

注意事项

- 不要跳过人工审核环节，否则噪声会逐轮累积
- 验证集必须独立且全人工标注，不能使用模型预测的标签
- 迭代 3-5 轮后收益递减，应适时停止并评估是否需要其他策略

第六章 LLM 时代的信息抽取

大语言模型（LLM）的兴起为信息抽取带来了新的范式。本章对比 BERT 微调方案与 LLM 方案的优劣，并给出实际项目中的推荐策略与混合方案。

LLM（Large Language Model，大语言模型）是指参数量巨大（通常数十亿以上）、在海量文本上预训练的通用语言模型，如 GPT、Claude、LLaMA 等。它们具备强大的语言理解和生成能力，可通过 Prompt 直接执行各类 NLP 任务。

6.1 BERT 微调序列标注方案

模型架构

BERT 微调方案的经典架构：

输入文本 → Token 序列
↓
BERT Encoder（双向 Transformer）
↓
Token-level Linear Layer
↓
CRF 层（可选）
↓
BIO 标签序列输出

- 输入层**：将文本转化为 Token 序列（**WordPiece** 分词，一种子词分词方法，将词拆分为更小的子词单元，如 "playing" → "play" + "##ing"，以解决 OOV 问题）
- BERT Encoder**：通过双向 **Transformer**（一种基于自注意力机制的深度学习架构，能并行处理序列中所有位置，捕捉全局依赖关系）编码上下文信息，输出每个 token 的向量表示
- Linear Layer**：将每个 token 的表示映射到标签空间的得分（emission score）
- CRF 层（可选）**：建模标签间的转移约束，输出全局最优的标签序列
- 输出**：BIO/BIOES 标签序列

优势

优势	说明
精度高	特别是领域数据充足时，BERT + CRF 可以达到很高的 F1
推理速度快	毫秒级延迟，支持大规模批处理
输出格式可控	严格输出 BIO 序列，格式确定性强
部署成本低	小模型可蒸馏/量化（蒸馏 = Knowledge Distillation，用大模型指导小模型学习；量化 = Quantization，将模型参数从高精度(如FP32)转为低精度(如INT8)，二者均旨在减小模型体积和推理开销），CPU 即可推理

劣势

劣势	说明
需要标注数据	冷启动难，需要一定量的标注数据才能训练
标签类型固定	新增实体类型需要重新标注和训练
难以处理嵌套结构	标准 BIO 方案无法表示嵌套实体

6.2 LLM 信息抽取方案

LLM 为信息抽取提供了三种主要方式：

方式 1：Prompt 直接抽取

原理：使用 **Zero-shot**（零样本，不给模型任何示例，直接描述任务让模型执行） / **Few-shot**（少样本，给模型几个示例后再让它执行任务）提示词，直接指定抽取目标和格式。

🕒 **新手类比**：Zero-shot 就像你招聘了一个新员工，只告诉他"你的任务是识别文本中的人名"就让他开始工作。Few-shot 是你先给他看了三个正确的例子，再让他做。显然 Few-shot 靠谱得多，但 Zero-shot 能快速验证"这个任务到底能不能做"。

Zero-shot 示例 (不给例子)

System Prompt

你是一个命名实体识别助手。请从以下文本中抽取所有人名(**PER**)、地名(**LOC**)、机构名(**ORG**)。以 **JSON** 数组格式输出，每个实体包含 **text** (原文)、**type** (类型)、**start** (起始位置) 和 **end** (结束位置)。如果没有实体，返回空数组 **[]**。

输入：马云在杭州创立了阿里巴巴集团。

输出: [{"text": "马云", "type": "PER", "start": 0, "end": 2},
{"text": "杭州", "type": "LOC", "start": 3, "end": 5},
{"text": "阿里巴巴集团", "type": "ORG", "start": 8, "end": 14}]

Few-shot 示例 (给 3 个例子)

System Prompt

你是一个 **NER** 标注助手。以下是 3 个标注示例，请按照相同格式 | 对后面的新句子进行标注。

示例1:

输入: 比尔盖茨创立了微软。

输出: [{"text": "比尔盖茨", "type": "PER"}, {"text": "微软", "type": "ORG"}]

示例2:

输入: 库克参观了位于北京的苹果总部。

输出: [{"text": "库克", "type": "PER"}, {"text": "北京", "type": "LOC"}, {"text": "苹果", "type": "ORG"}]

示例3:

输入: 今天天气很好。

输出: []

现在请对以下句子进行标注:

输入：马云在杭州创立了阿里巴巴。

输出: [{"text": "马云", "type": "PER"},
{"text": "杭州", "type": "LOC"},
{"text": "阿里巴巴", "type": "ORG"}]

新手提示： Few-shot 示例的质量和多样性直接影响效果。示例要覆盖"有实体的句子"和"没有实体的句子"两种情况，否则 LLM 会倾向于"强行找出实体"。

优势	劣势
无需任何标注数据	一致性差，边界容易出错
快速验证可行性	不同 prompt 格式结果差异大

方式 2：结构化输出

原理：通过 **Function Calling**（函数调用，LLM 的一种能力，允许模型根据用户定义的函数接口生成结构化参数，而非自由文本）/ JSON Schema，强制模型输出结构化结果。

完整的 Function Calling 示例（伪代码）

```
# 定义函数接口 —— 告诉 LLM "你必须按这个格式输出"
functions = [{
    "name": "extract_entities",
    "description": "从输入文本中抽取命名实体",
    "parameters": {
        "type": "object",
        "properties": {
            "entities": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "text": {"type": "string", "description": "实体原文"},
                        "type": {
                            "type": "string",
                            "enum": ["PER", "LOC", "ORG"],
                            "description": "实体类型"
                        }
                    }
                },
                "start": {"type": "integer", "description": "起始位置"},
                "end": {"type": "integer", "description": "结束位置"}
            },
            "required": ["text", "type", "start", "end"]
        }
    }
}]

# 调用 LLM
response = llm.chat(
    messages=[{"role": "user", "content": "马云在杭州创立阿里巴巴。"}],
    functions=functions,
    function_call={"name": "extract_entities"} # 强制调用此函数
)

# LLM 必须返回严格符合 schema 的结构
# 输出: {"entities": [
#     {"text": "马云", "type": "PER", "start": 0, "end": 2},
#     {"text": "杭州", "type": "LOC", "start": 3, "end": 5},
#     {"text": "阿里巴巴", "type": "ORG", "start": 6, "end": 10}
# ]}
```

新手提示：Function Calling 的核心优势是**格式保障**——与传统文本输出不同，LLM 被“锁死”在你的 schema 框架内，不会输出 “人名：马云” 这种后处理困难的自由文本。

优势	劣势
格式可靠，利于下游处理	仍依赖 LLM，成本较高
减少后处理工作量	某些 LLM 不支持 Function Calling

方式 3：指令微调（SFT）

原理：在 IE（信息抽取）数据集上做 **SFT**（*Supervised Fine-Tuning*，有监督微调，用特定格式的指令-输出数据对继续训练 LLM，使其学会按指定格式输出），使模型输出特定格式的抽取结果。

🎯 **新手类比：**Zero-shot 是给新人看岗位说明书就让他干活，Few-shot 是给新人看 3 个范例再干活，**SFT 是让新人看 10000 个范例并做模拟练习**，直到他形成“肌肉记忆”。SFT 效果最好但也最昂贵。

SFT 训练数据格式示例（每条数据是一个 Instruction-Output 对）

```
// 数据样例1：简单实体识别
{
  "instruction": "识别以下文本中的人名(PER)、地名(LOC)、机构名(ORG)，输出JSON格式。",
  "input": "马云在杭州创立了阿里巴巴。",
  "output": "{\"entities\": [{\"text\": \"马云\", \"type\": \"PER\"}, {\"text\": \"杭州\", \"type\": \"LOC\"}, {\"text\": \"阿里巴巴\", \"type\": \"ORG\"}]}"
}

// 数据样例2：医疗领域实体
{
  "instruction": "提取以下病历文本中的疾病(DIS)、药品(DRUG)、症状(SYM)实体。",
  "input": "患者因发热、咳嗽三天，服用布洛芬后体温降至正常。",
  "output": "{\"entities\": [{\"text\": \"发热\", \"type\": \"SYM\"}, {\"text\": \"咳嗽\", \"type\": \"SYM\"}, {\"text\": \"布洛芬\", \"type\": \"DRUG\"}]}"
}

// 数据样例3：金融领域（实体 + 关系）
{
  "instruction": "从以下财经新闻中提取公司实体，并标注其关联事件。",
  "input": "腾讯宣布以10亿美元收购搜狗剩余股份。",
  "output": "{\"entities\": [{\"text\": \"腾讯\", \"type\": \"ORG\", \"role\": \"收购方\"}, {\"text\": \"搜狗\", \"type\": \"ORG\", \"role\": \"被收购方\"}, {\"text\": \"10亿美元\", \"type\": \"AMOUNT\", \"role\": \"收购金额\"}]}"
}
```

优势	劣势
兼顾灵活性与一致性	需要指令格式训练数据
输出质量高于纯 Prompt	微调成本和训练数据准备成本

Zero-shot / Few-shot / SFT 效果速查

方式	标注数据需求	输出质量	适用场景	成本
Zero-shot	0 条	★ ★	快速验证、原型探索	低
Few-shot	3~10 条	★ ★ ★	小规模应用、PoC	低
SFT	1000+ 条	★ ★ ★ ★ ★	生产级部署	高（训练资源）
BERT微调	1000+ 条	★ ★ ★ ★ ★	高吞吐、低延迟生产	中（推理便宜）

6.3 方案对比

对比维度	BERT 微调 + CRF	LLM Prompt / SFT
标注数据需求	高（需专项标注）	低（零样本可用）
灵活性（新类型）	低（需重训）	高（改 Prompt 即可）
推理速度	快（毫秒级）	慢（秒级）
部署成本	低（小模型可蒸馏）	高（GPU 密集）
输出一致性	强（格式固定）	较弱（需后处理）
嵌套/复杂结构	弱（需特殊设计）	强（自然语言理解）
冷启动场景	难	好（zero/few-shot）

关键洞察

- **BERT 方案**在"数据充足 + 追求效率"的场景下占优
- **LLM 方案**在"冷启动 + 灵活性"的场景下占优
- 两者并非对立关系，而是互补关系

6.4 推荐策略与混合方案

选择决策树

数据充足？
├─ 是 → 追求低延迟、高吞吐？
│ └─ 是 → BERT + CRF
│ └─ 否 → 类别是否频繁变更？
│ └─ 否 → BERT + CRF
│ └─ 是 → LLM
└─ 否 → 冷启动 / 低资源
→ LLM

适用场景

BERT + CRF 适用于：

- 拥有 1000+ 条专项标注数据
- 追求低延迟、高吞吐的生产部署

- 类别稳定，不频繁变更

LLM 适用于：

- 新业务场景、无历史标注
- 需快速验证 NER 可行性
- 类别多变或结构复杂（如关系抽取、事件抽取）

混合方案（推荐）

混合方案结合两者优势，分阶段推进：

LLM 伪标注 → 人工审核 → 种子集训练 BERT → 上线推理 → 滚雪球扩充

伪标注（Pseudo-labeling）：用模型对无标注数据进行预测，将预测结果作为"伪标签"用于训练。这些标签不是人工标注的，可能包含噪声，但可以作为训练数据的补充。

详细流程：

1. **LLM 伪标注**：使用 LLM 对无标注文本生成初始标注
2. **人工审核 + 纠错**：对 LLM 输出进行人工审核，修正错误后形成**种子集**（Seed Set，经过验证的高质量初始训练数据集，是后续迭代扩充的基础）
3. **种子集训练 BERT**：用审核后的数据训练 BERT 小模型
4. **上线推理**：BERT 小模型上线，提供低延迟、高吞吐的推理服务
5. **滚雪球扩充**：利用 BERT 模型对更多无标注文本预测，高置信样本经人工审核后加入训练集

混合方案的优势

阶段	工具	作用
冷启动	LLM	快速生成伪标注，解决标注数据从 0 到 1 的问题
数据扩充	LLM + 人工	LLM 生成候选，人工审核保证质量
生产部署	BERT	低延迟、低成本的推理服务
持续优化	滚雪球	数据越多模型越好，形成正向循环

实践建议

1. **冷启动阶段**：先用 LLM 零样本/少样本抽取，快速验证 NER 在当前业务场景的可行性
2. **种子集构建**：LLM 生成 500-1000 条伪标注 + 人工审核，保证种子集质量
3. **模型训练**：用种子集训练 BERT + CRF，评估 F1 是否满足业务需求
4. **迭代优化**：利用滚雪球策略持续扩充数据，逐步提升模型性能
5. **生产部署**：BERT 模型量化后部署，满足延迟和吞吐要求

6.5 总结

场景	推荐方案
数据充足 + 生产部署	BERT + CRF
冷启动 / 低资源	LLM Prompt / SFT
最佳实践	LLM 伪标注 → 人工审核 → BERT 训练 → 滚雪球

LLM 灵活冷启动，BERT 精准高效，混合方案最优。

混合方案的核心思想是**各取所长**：LLM 负责灵活的冷启动和数据扩充，BERT 负责高效的生产部署，两者通过滚雪球迭代形成正向循环。

附录：术语表

下表汇总了本文档中出现的专业术语和缩写，供新手查阅参考。

术语	全称	含义
Token	—	文本的最小处理单元，可以是字、词或子词，取决于分词方式
OOV	Out-of-Vocabulary	未登录词，即训练词典中未收录的词，模型未见过
Span	—	文本中一段连续的 token 序列，是序列标注输出的基本单位
BIO	Begin-Inside-Outside	最常用的 NER 标注体系，B=实体起始，I=实体内部，O=非实体
BIOES	Begin-Inside-Outside-End-Single	BIO 的扩展，增加 E(实体结束) 和 S(单字实体) 标签
NER	Named Entity Recognition	命名实体识别，从文本中识别具有特定意义的实体
POS	Part-of-Speech	词性标注，为每个词分配语法类别
SRL	Semantic Role Labeling	语义角色标注，标注谓词-论元结构
CRF	Conditional Random Field	条件随机场，对标签序列联合建模的概率图模型
BERT	Bidirectional Encoder Representations from Transformers	基于双向 Transformer 的预训练语言模型
LSTM	Long Short-Term Memory	长短期记忆网络，一种循环神经网络变体
Transformer	—	基于自注意力机制的深度学习架构，BERT/GPT 的基础
Seq2Seq	Sequence-to-Sequence	序列到序列任务，输入输出长度可以不同（如机器翻译）
WordPiece	—	子词分词方法，将词拆分为更小的子词单元以解决 OOV
Softmax	—	归一化指数函数，将实数向量转为概率分布
Linear Layer	—	全连接层，将高维向量映射到目标维度
Emission Score	—	发射分数，当前 token 属于某标签的局部得分
Transition Score	—	转移分数，相邻标签之间的转移得分
Viterbi	—	维特比算法，用动态规划寻找最优标签序列

术语	全称	含义
Dynamic Programming	—	动态规划，将复杂问题分解为重叠子问题递推求解
Forward Algorithm	—	前向算法，高效计算配分函数的动态规划算法
Partition Function	—	配分函数，对所有可能状态的得分求和，用于归一化
Receptive Field	—	感受野，模型能"看到"的输入范围
Exact Match	—	精确匹配，实体的起止边界与类型完全一致才算正确
Precision	—	精确率，预测为正例中真正为正例的比例
Recall	—	召回率，真正正例中被正确预测的比例
F1	—	精确率和召回率的调和平均
TP / FP / FN	True Positive / False Positive / False Negative	真正例 / 假正例 / 假负例
Harmonic Mean	—	调和平均，对较小值更敏感的平均方式
Fine-tuning	—	微调，在预训练模型基础上用下游任务数据继续训练
Cold Start	—	冷启动，在没有任何历史数据或标注的情况下从零开始
Distant Supervision	—	远程监督，用已有知识库/词典自动生成标注
Pseudo-labeling	—	伪标注，用模型预测结果作为训练标签
Seed Set	—	种子集，经过验证的高质量初始训练数据集
Confidence	—	置信度，模型对预测结果的可信程度
Inference Latency	—	推理延迟，模型做出一次预测所需的时间
Batch Processing	—	批处理，同时处理多个样本以提高吞吐量
Distillation	Knowledge Distillation	知识蒸馏，用大模型指导小模型学习以压缩模型

术语	全称	含义
Quantization	—	量化，将模型参数从高精度转为低精度以减少计算量
LLM	Large Language Model	大语言模型，参数量巨大的通用预训练语言模型
Zero-shot	—	零样本学习，不给示例直接让模型执行任务
Few-shot	—	少样本学习，给少量示例后让模型执行任务
SFT	Supervised Fine-Tuning	有监督微调，用指令格式数据继续训练 LLM
Function Calling	—	函数调用，LLM 根据函数接口生成结构化参数
CoNLL	Conference on Computational Natural Language Learning	计算自然语言学习会议，其标注标准被广泛使用
PKU	Peking University	北京大学，其词性标注体系是中文标准之一
Cohen's Kappa	—	衡量两个标注者间一致性的统计指标，>0.8 表示良好
Arc-Eager	—	一种增量式依存句法解析策略
Span-based	—	枚举所有可能 span 并独立分类的方法，天然支持嵌套
IE	Information Extraction	信息抽取，从非结构化文本中提取结构化信息